

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

LAB 3 — MINI PROJECT

Remote Patient Vitals Alert & Monitoring App

Healthcare & Telemedicine

Student	CHIRANTHAN HG
SRN	PES1UG24CS564
Problem Statement	16
Version	1.0
Status	Final Submission
Date	26-09-2026

Architecture, design, security, API and test-plan update

1. Introduction

This Software Architecture and Design Specification defines the architecture and detailed design of the Remote Patient Vitals Alert & Monitoring App. The system records vital signs, checks them against configured thresholds, alerts an on-call caregiver when abnormal readings are detected, and supports alert acknowledgement.

The design baseline contains five functional requirements: recording/submission, caregiver viewing, abnormality detection, caregiver notification and alert acknowledgement. It also contains two non-functional requirements: alert generation within 5 seconds and at least 99% service availability.

1.1 Purpose

Provide a secure, modular and maintainable design for continuous remote patient monitoring and timely caregiver response.

1.2 Scope

- Capture heart rate, blood pressure, SpO2 and body temperature.
- Validate and store readings with patient ID and timestamp.
- Compare readings against configured thresholds.
- Notify the on-call caregiver about abnormal readings.
- Display current/history and allow alert acknowledgement.

1.3 Actors

Actor	Responsibility
Remote Patient	Provides or submits vital readings.
Vital Signs Device	Provides measured vital values.
On-Call Caregiver	Views readings, receives alerts and acknowledges them.

2. Design Basis and Traceability

2.1 Requirements Baseline

ID	Requirement	Priority	Design element
FR-001	Record/submit HR, BP, SpO2, temperature	High	Patient UI + Device Adapter + Vitals API
FR-002	View latest and historical readings	High	Caregiver Dashboard + Query API
FR-003	Detect threshold violations	High	Alert & Threshold Engine
FR-004	Notify caregiver of abnormal reading	High	Notification Service
FR-005	Acknowledge active alert	Medium	Alert Management API + Alert DB
NFR-001	Alert within 5 sec in ≥95/100 cases	High	Event-driven alert path + monitoring
NFR-002	≥99% scheduled availability	High	Health checks + redundancy + monitoring

2.2 Use-Case Flow

Main flow: application opens → current readings are entered/received → readings are validated → stored with patient ID and timestamp → thresholds are checked → normal readings are displayed or an abnormal alert is generated → caregiver is notified → caregiver may acknowledge the alert.

3. Architecture

3.1 Goals and Constraints

- Security of patient and authentication data.
- Reliable monitoring during transient failures.
- Fast alert generation to meet the 5-second requirement.
- Separation of UI, API, business rules, alerting and persistence.
- Healthcare-data sensitivity and device/network dependency.

3.2 Stakeholders and Concerns

Stakeholder	Concern
Remote Patient	Simple, reliable submission
On-Call Caregiver	Fast readings and actionable alerts
Developer	Modularity and maintainability
QA/Security	Traceability, validation and measurable performance

3.3 Component UML Diagram

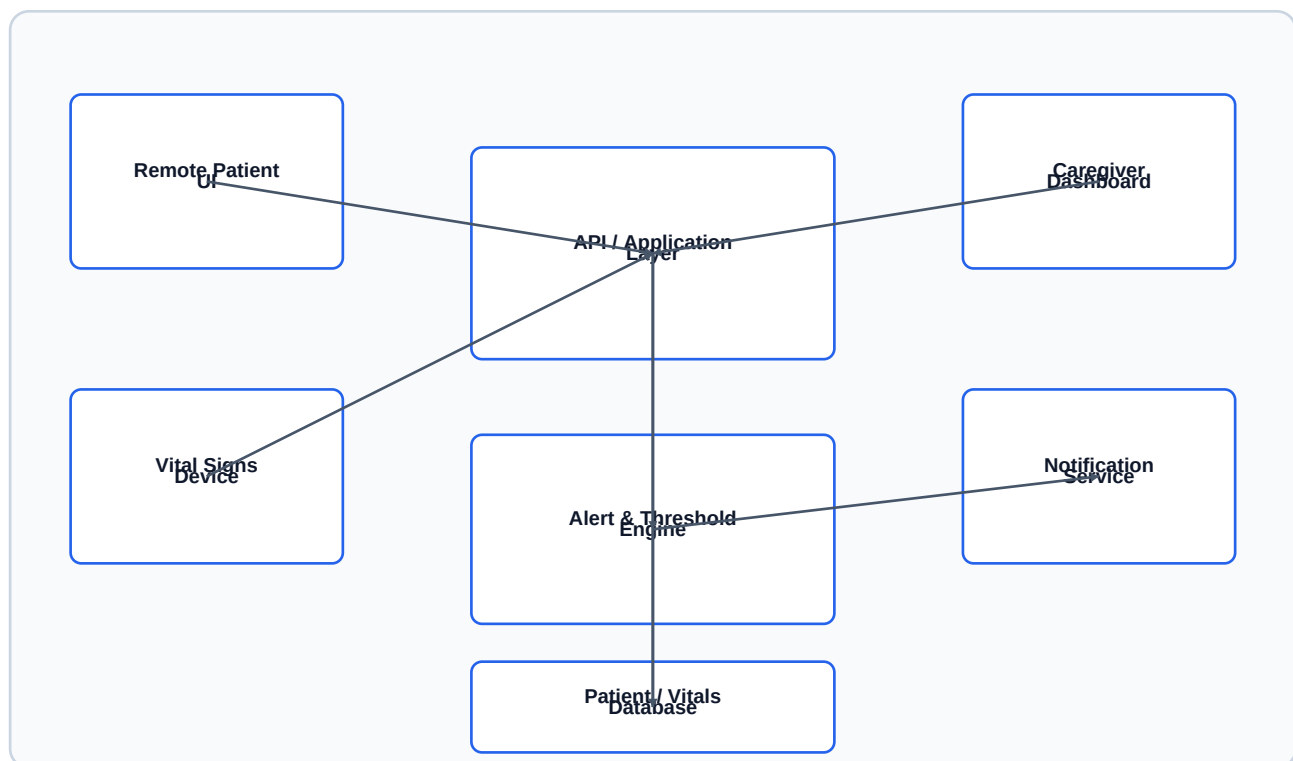


Figure 1. Component architecture.

3.4 Component Descriptions

Component	Responsibility
Remote Patient UI	Collects readings and displays validation/submission status.
Vital Signs Device Adapter	Normalizes measurements received from a device.
Caregiver Dashboard	Displays latest/history and active alerts; supports acknowledgement.
API / Application Layer	Authenticates, validates, coordinates services and persistence.
Alert & Threshold Engine	Applies thresholds, classifies abnormal readings and creates alert events.
Notification Service	Delivers abnormal-reading notifications.
Patient/Vitals Database	Stores patients, readings, thresholds, alerts and acknowledgement timestamps.

3.5 Architecture Pattern and Rationale

Layered architecture with event-driven alert handling is selected. Presentation, API/application, business rules, alerting and persistence are separated. Threshold violations produce alert events so notification processing can be decoupled from the recording path.

3.6 Technology Stack and Data Stores

Layer	Technology / approach
Client	Web or mobile-responsive UI
Backend	Java + Spring Boot REST APIs
Database	MySQL / relational database
Communication	HTTPS/TLS + JSON REST
Monitoring	Health checks, logs and alert-latency metrics

3.7 Risks and Mitigations

Risk	Mitigation
Network failure	Retry queues, health checks and redundant paths.
Invalid/malicious input	Strict validation, bounds checking and rate limiting.
Unauthorized access	Authentication and role-based access control.
Sensitive logs	Never log credentials or unnecessary sensitive patient data.
Notification failure	Persistent alert events, retries and monitoring.

3.8 Traceability

Requirement	Mapped design
FR-001	Patient UI + Device Adapter + Vitals API + DB
FR-002	Caregiver Dashboard + Query API + DB
FR-003	Threshold Engine
FR-004	Alert Event + Notification Service
FR-005	Alert Management API + Alert DB
NFR-001	Event-driven path + latency monitoring
NFR-002	Health checks + redundancy + monitoring

3.9 Security Architecture

- Authenticate all patient and caregiver sessions.
- Use role-based access control for caregiver-only history and acknowledgement actions.
- Use TLS for client/API communication.
- Validate vital values, identifiers and timestamps before persistence.
- Audit alert creation and acknowledgement without logging credentials.
- Apply rate limiting and operational monitoring to exposed endpoints.

4. Design

4.1 Design Overview

REST APIs provide client interaction while a relational database stores patient, vital, threshold and alert information. Business rules are centralized so the same validation and threshold logic applies to UI and device-originated readings.

4.2 UML Sequence Diagram 1 — Record and Submit Vital Signs

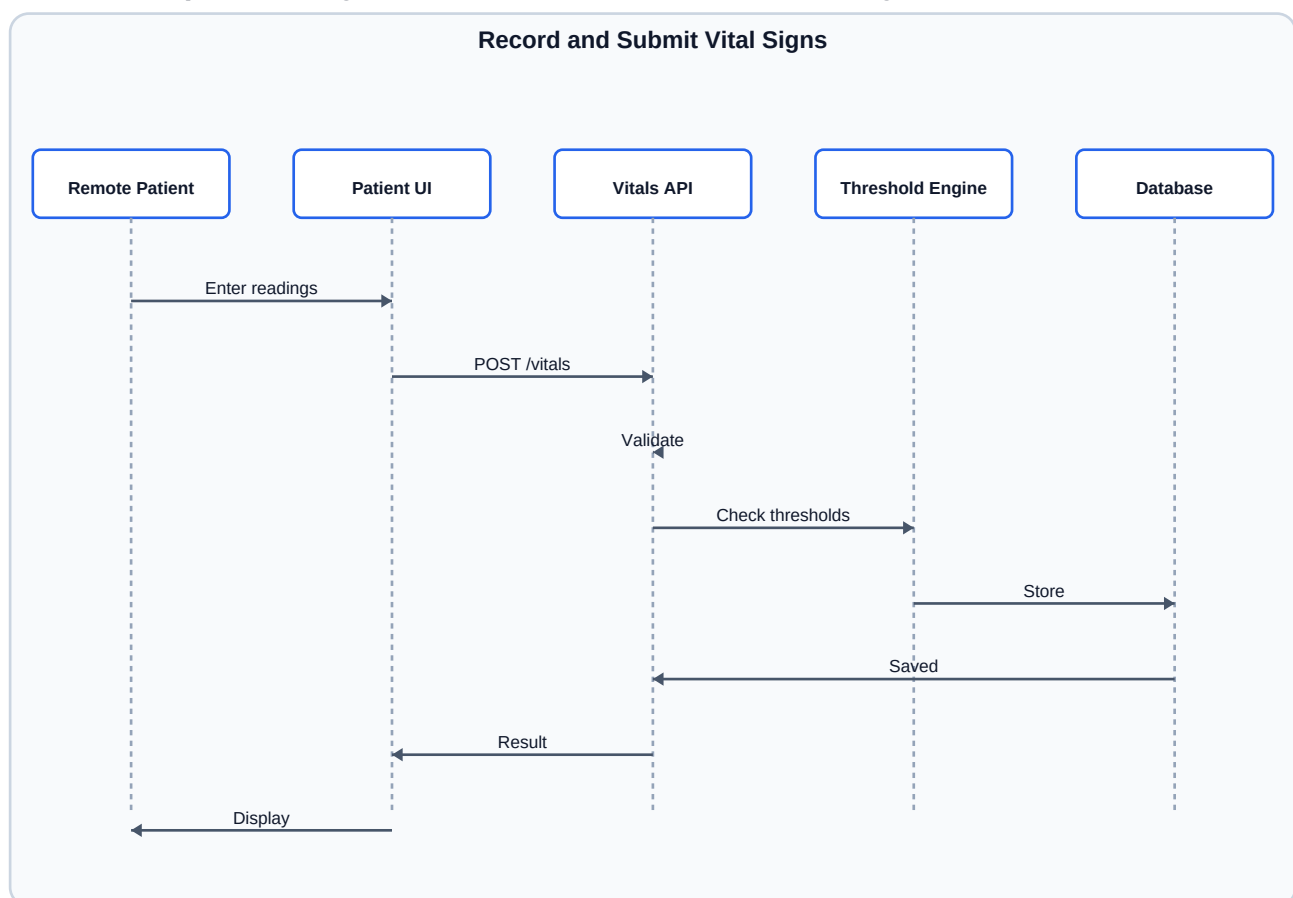


Figure 2. Valid vital-sign submission flow.

4.2 UML Sequence Diagram 2 — Alert and Acknowledgement

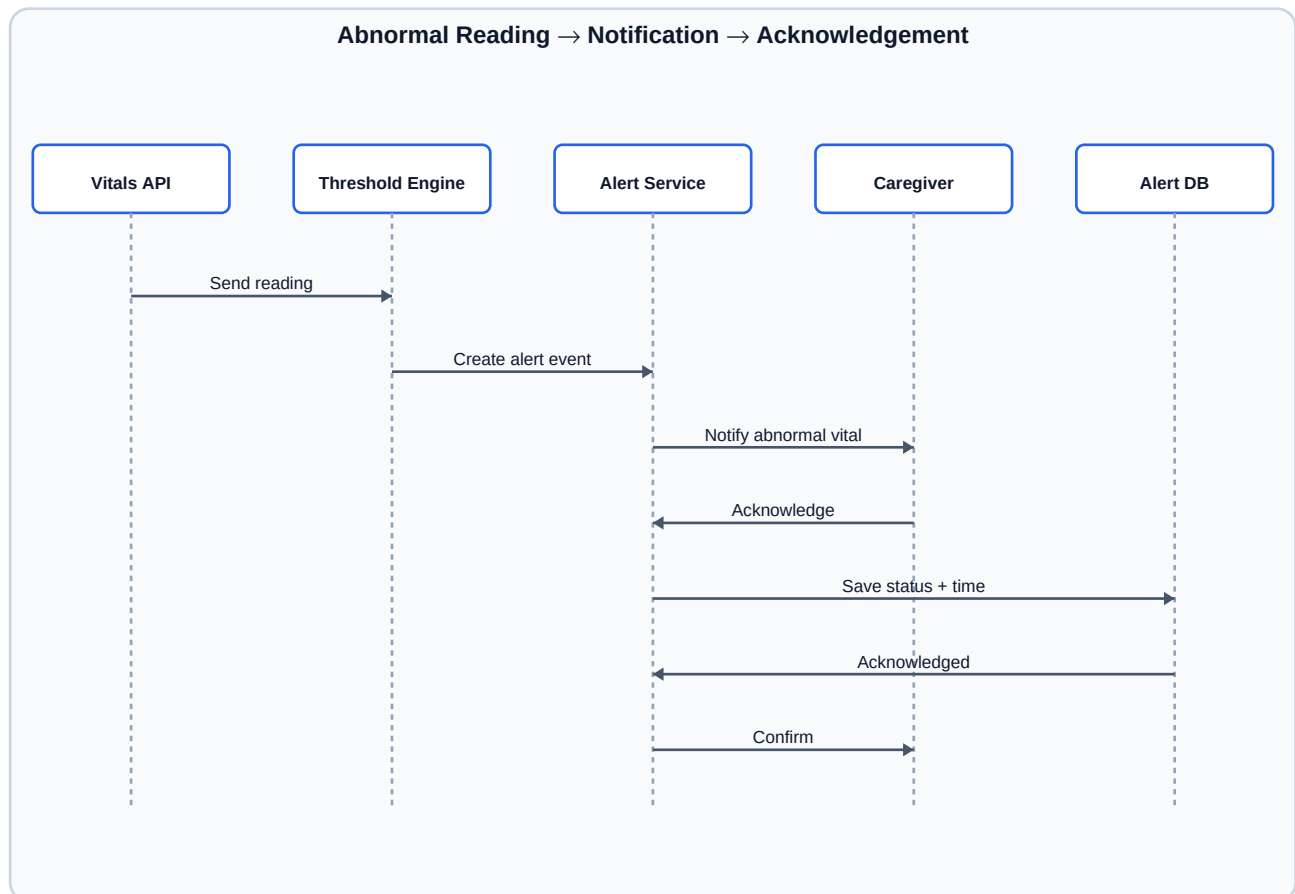


Figure 3. Abnormal-reading and caregiver acknowledgement flow.

4.3 API Design

Vitals API — POST `/api/v1/vitals`

Request: {patientId, heartRate, systolicBP, diastolicBP, spo2, temperature, timestamp}

Response: {readingId, status, abnormal, timestamp}

Errors: 400 invalid reading; 401 unauthorized; 404 patient not found.

Alert Management API — PATCH `/api/v1/alerts/{alertId}/acknowledge`

Request: {caregiverId}

Response: {alertId, status: "Acknowledged", acknowledgedAt}

Errors: 401 unauthorized; 403 not permitted; 404 alert not found; 409 already acknowledged.

4.4 Error Handling, Logging and Monitoring

- Use standardized HTTP status codes and machine-readable error responses.
- Reject invalid ranges and missing mandatory fields before persistence.
- Do not expose stack traces or sensitive patient information to clients.
- Log correlation ID, event type, timing and outcome; exclude credentials.
- Monitor alert latency, API errors, database health, notification delivery and availability.

4.5 UX Design

- Patient screen provides clear inputs for HR, BP, SpO2 and temperature.
- Validation messages identify invalid measurements.
- Caregiver dashboard prioritizes active abnormal alerts and timestamped readings.
- Acknowledgement is a single explicit action with confirmation.
- Use readable typography, accessible contrast and clear status labels.

4.6 Architecture Decision Record

Decision	Choice	Reason
Architecture	Layered + event-driven	Separation of concerns and fast alert propagation.
Database	Relational	Structured consistency for readings, alerts and timestamps.
API	REST/JSON	Interoperability for web/mobile/device clients.
Security	TLS + RBAC + validation	Protect data and restrict operations.

4.7 Open Issues and Next Steps

- Integrate real medical devices after interface and calibration validation.
- Add configurable threshold administration with audit history.
- Evaluate escalation channels for unacknowledged critical alerts.
- Add MFA for caregiver accounts.

5. Test Plan Update

This section adds eight test cases covering both functional and non-functional requirements, satisfying the mini-project requirement for 7–10 test cases.

TC ID	Req	Test case	Expected result
TC-01	FR-001	Submit valid HR, BP, SpO2 and temperature	Stored with patient ID/timestamp; success returned.
TC-02	FR-001	Submit invalid SpO2 or missing field	Validation error; invalid record not stored.
TC-03	FR-002	Caregiver opens patient history	Latest and historical readings are displayed.
TC-04	FR-003	Submit reading crossing threshold	Reading marked abnormal; alert event created.
TC-05	FR-004	Trigger abnormal reading	Notification contains patient, vital, value and timestamp.
TC-06	FR-005	Acknowledge active alert	Status becomes Acknowledged; time recorded.
TC-07	NFR-001	Run 100 abnormal-reading alert tests	At least 95/100 alerts complete within 5 seconds.
TC-08	NFR-002	Measure scheduled service availability	Availability is at least 99%.

5.1 Entry and Exit Criteria

- Entry: stable build, configured test environment, valid test data and required services available.
- Exit: planned cases executed, no critical defects open, acceptance criteria satisfied and NFR measurements recorded.

5.2 Security Validation

- Verify caregiver authorization for history and acknowledgement operations.
- Verify TLS is used for client/API communication.
- Verify credentials and sensitive patient data are absent from logs.
- Test malformed inputs and endpoint rate limiting.

6. Submission Checklist

Deliverable	Status
Architecture goals/constraints/stakeholders	Completed
Component UML diagram + descriptions	Completed
Architecture pattern + rationale	Completed
Technology stack + data stores	Completed
Risks + mitigations	Completed
Requirement traceability	Completed
Security architecture	Completed
At least 2 UML sequence diagrams	Completed
API design for 2 components	Completed
Error handling/logging/monitoring	Completed
UX + open issues	Completed
7–10 test cases	Completed — 8 cases

6.1 References

- Lab 1 Requirements Table — Remote Patient Vitals Alert & Monitoring App, PES1UG24CS564.
- Lab 1 Use-Case Flow — UC-01 Record and Monitor Vital Signs.
- Software Architecture and Design Specification template supplied for the mini project.
- Mini-project deliverables specification for Architecture & Design and Test Plan update.

Role	Name	Signature / Date
Student / Developer	CHIRANTHAN HG	_____
QA / Reviewer	_____	_____
Faculty / Evaluator	_____	_____

END OF LAB 3 SUBMISSION DOCUMENT